React Native y Figma aplicados al proyecto Rotula

Gerson Borja (carné: [número de carné])

[Nombre completo del segundo integrante] (carné: [número de carné])

[Escuela o facultad], [Nombre de la universidad]

[Código y nombre de la materia] · Grupo: [número de grupo]

Docente: [Nombre del docente]

[Fecha de entrega: 18 de octubre de 2026]

React Native y Figma aplicados al proyecto Rotula

Introducción

Rotula es una aplicación web que permite a tiendas y negocios pequeños crear los rótulos de sus ofertas y descargarlos en un PDF listo para imprimir, con cuatro afiches por hoja. La versión web está hecha con Next.js, guarda los datos en una base MySQL y está publicada en Vercel (https://rotula.vercel.app). Para el Avance 1 de la materia se construyó Rotula móvil, una aplicación para Android e iOS hecha con React Native y Expo que usa la misma API y la misma base de datos que la web.

El avance pide cuatro entregables técnicos: un inicio de sesión y registro funcionales, un módulo principal con navegación, un diseño coherente con mockups hechos en Figma y validaciones en los formularios. Este documento reúne la investigación sobre React Native y Figma y explica cómo se aplicó cada herramienta en el proyecto. También sirve como documentación técnica del proceso de diseño UX/UI.

El documento tiene cuatro partes. El marco teórico explica los conceptos de React Native y de Figma que se usaron. Luego se describe la arquitectura de la aplicación móvil, su navegación, sus validaciones y su conexión con la API web. Por último, se documenta el proceso de diseño en Figma y se presentan las conclusiones.

Los entregables del avance están en estos enlaces:

- Repositorio del proyecto en GitHub: [pega aquí el enlace del repositorio]
- Video de demostración: [pega aquí el enlace del video]
- Mockups y prototipo en Figma: https://www.figma.com/design/TbgoZj5OUNLUFCJI2Mv3hF
- Versión web de Rotula: https://rotula.vercel.app

Marco teórico

React Native

Qué es React Native

React Native es un framework para crear aplicaciones de Android e iOS con JavaScript y React. Con él, los componentes de React llaman desde JavaScript a las vistas nativas de cada sistema. En tiempo de ejecución, React Native crea la vista de Android o de iOS que corresponde a cada componente, por lo que la aplicación se ve y rinde como cualquier otra aplicación nativa (Meta Platforms, 2026a).

Este enfoque es parte del desarrollo multiplataforma, que busca mantener una sola base de código para varios sistemas operativos. Biørn-Hansen et al. (2018) señalan que desarrollar para móviles es complejo por la fragmentación de dispositivos y plataformas. Su revisión de la investigación sobre el tema pone énfasis en conceptos centrales como la experiencia de usuario, el acceso a las funciones del dispositivo, el rendimiento y la seguridad.

Componentes nativos y componentes básicos

En Android y en iOS, una vista es el bloque básico de la interfaz: un rectángulo que muestra contenido o responde a lo que hace el usuario (Meta Platforms, 2026a). React Native trae un conjunto de componentes básicos (*Core Components*) que se convierten en la vista nativa de cada plataforma. La Tabla 1 muestra los que usa Rotula móvil.

Tabla 1

Componentes básicos de React Native usados en Rotula móvil

Componente de React Native	Vista en Android	Vista en iOS	Uso en Rotula móvil
`<View>`	ViewGroup	UIView	Contenedores, tarjetas y filas
`<Text>`	TextView	UITextView	Todos los textos, a través del componente `Texto`
`<ScrollView>`	ScrollView	UIScrollView	Pantallas Inicio, Crear afiches y Perfil
`<TextInput>`	EditText	UITextField	Campos de los formularios, a través del componente `Campo`

Nota. Las equivalencias nativas son de Meta Platforms (2026a). La última columna describe el código del proyecto.

JSX

JSX es una sintaxis que permite escribir los elementos de la interfaz dentro de JavaScript (Meta Platforms, 2026c). Se parece a HTML, pero es más estricta: toda etiqueta se cierra, varios elementos necesitan un solo elemento raíz y la mayoría de los atributos se escriben en *camelCase* (Meta Platforms, s.f.-c). Las llaves insertan valores de JavaScript en el marcado; por ejemplo, la tarjeta de un documento muestra su título con `{documento.titulo}`.

Props

Las props son los datos que un componente padre le pasa a un componente hijo para configurarlo (Meta Platforms, 2026c). En Rotula móvil, el componente `Boton` recibe las props `titulo`, `variante`, `icono` y `alPresionar`. Así, el mismo componente produce el botón naranja «Iniciar sesión» (`variante="acento"`) y el botón transparente «Ver historial» (`variante="fantasma"`).

Estado y contexto

El estado es la memoria de un componente: guarda datos que cambian con el tiempo y, cada vez que cambia, el componente se vuelve a dibujar. Se crea con el *hook* `useState`, y la regla general es usar props para configurar y estado para los datos que cambian (Meta Platforms, 2026c). En la pantalla Login, `useState` guarda el correo, la contraseña, los errores de validación y si la petición está en curso.

Cuando muchos componentes necesitan el mismo dato, React ofrece el contexto. Con él, un componente padre pone información a disposición de todo el árbol que tiene debajo, sin pasarla por props en cada nivel (Meta Platforms, s.f.-b). Rotula móvil guarda en un contexto al usuario que inició sesión.

Ciclo de vida y efectos

Un componente se monta cuando aparece en pantalla, se actualiza cuando recibe nuevas props o un nuevo estado y se desmonta cuando se quita (Meta Platforms, s.f.-a). Los efectos, creados con `useEffect`, tienen un ciclo propio: empiezan a sincronizar el componente con un sistema externo y después dejan de hacerlo con una función de limpieza. En Rotula móvil, el proveedor de sesión usa `useEffect` con una lista de dependencias vacía para consultar al servidor si hay una sesión activa cuando se abre la aplicación.

React Navigation

React Navigation es una biblioteca de navegación para React Native y la web. Sus navegadores manejan las transiciones entre pantallas y dan elementos comunes como encabezados y barras de pestañas (React Navigation, s.f.-c). El navegador de pila nativo (*native stack*) coloca cada pantalla nueva encima de la anterior y deja las transiciones y los gestos a las

API de cada sistema: `UINavigationController` en iOS y `Fragment` en Android (React Navigation, s.f.-d).

El navegador de pestañas inferiores (*bottom tabs*) muestra una barra en la parte de abajo para cambiar de pantalla. Cada pestaña define su ícono con `tabBarIcon` y el color de la pestaña activa con `tabBarActiveTintColor`; además, cada pantalla se monta la primera vez que se abre (React Navigation, s.f.-b). El *hook* `useFocusEffect` ejecuta un efecto cada vez que una pantalla recibe el foco (React Navigation, s.f.-e).

Para el inicio de sesión, la documentación recomienda definir las pantallas según el estado de autenticación. Cuando ese estado cambia, React Navigation quita las pantallas que ya no corresponden y muestra la primera que sí corresponde, sin llamar a `navigate` a mano. Así, el botón atrás no puede regresar a las pantallas de acceso (React Navigation, s.f.-a).

Expo y Expo Go

La documentación de React Native recomienda usar un framework para las aplicaciones nuevas, porque este ya resuelve funciones como la navegación y el acceso a las API nativas. El framework que recomienda es Expo (Meta Platforms, 2026b). Expo es un framework de código abierto para aplicaciones que funcionan de forma nativa en Android, iOS y la web; incluye el Expo SDK, que es un conjunto de módulos probados, la herramienta de línea de comandos Expo CLI, la aplicación Expo Go y los servicios en la nube EAS (Expo, 2026e).

Expo Go es una aplicación para el celular que abre un proyecto en desarrollo sin tener que compilarlo. Expo la presenta como un entorno de aprendizaje y un *sandbox* para empezar (Expo, s.f.). También aclara que tiene límites y que no sirve para proyectos de producción, que necesitan una *development build* (Expo, 2026b).

Figma

Qué es Figma

Figma Design es una herramienta para crear, compartir y probar diseños de sitios web, aplicaciones móviles y otros productos digitales. Está pensada para que diseñadores, gerentes de producto, redactores y desarrolladores participen en el mismo proceso. Sus archivos son espacios de trabajo colaborativos que pueden tener varias páginas, cada una con su propio lienzo (Figma, s.f.-i).

Marcos

Un marco (*frame*) es un contenedor del lienzo que sirve para organizar los diseños, parecido a una mesa de trabajo. La herramienta de marcos trae tamaños predefinidos para teléfonos, tabletas y computadoras, y un marco puede ir dentro de otro (Figma, s.f.-h). Los marcos también son la base del prototipado, que los conecta entre sí, y de la disposición automática.

Componentes y variantes

Un componente es un elemento reutilizable, como un botón o un ícono, que ayuda a mantener la coherencia del diseño. El componente principal define sus propiedades. Cada copia que se coloca en el diseño es una instancia que sigue vinculada al principal y recibe sus cambios (Figma, s.f.-e).

Las variantes agrupan componentes parecidos, como los tamaños o los estados de un botón, en un contenedor llamado conjunto de componentes (*component set*). Cada variante se describe con pares de propiedad y valor, por ejemplo `Size=Large`. En una instancia, la variante se cambia desde los menús de la barra lateral derecha (Figma, s.f.-c).

Variables y estilos

Las variables guardan valores reutilizables de cuatro tipos: color, número, texto (*string*) y booleano. Se agrupan en colecciones, y cada colección puede tener modos, como tema claro y tema oscuro, con un valor distinto por modo (Figma, s.f.-d). Los estilos de texto guardan una combinación tipográfica reutilizable: familia, peso, tamaño, interlineado y espaciado entre letras, entre otras propiedades (Figma, s.f.-b).

Disposición automática

La disposición automática (*auto layout*) hace que un marco acomode a sus elementos hijos y se ajuste cuando cambia el contenido. Los elementos fluyen en vertical, en horizontal o en cuadrícula. El relleno (*padding*) separa el contenido del borde del marco y el espacio (*gap*) separa un elemento de otro; además, cada elemento puede ajustarse a su contenido, llenar el contenedor o tener un tamaño fijo (Figma, s.f.-f).

Prototipado

Un prototipo es un conjunto de flujos interactivos que muestra cómo una persona recorrería los diseños (Figma, s.f.-g). Cada conexión une una zona activa, como un botón, con un marco de destino. La interacción tiene un desencadenante, que es lo que hace el usuario, y una acción, como navegar a otro marco. Las superposiciones (*overlays*) son marcos que aparecen encima de la pantalla actual y sirven para alertas o confirmaciones.

Un flujo agrupa los marcos conectados que forman una experiencia completa, como el registro de una cuenta. Cada flujo tiene un punto de inicio, y su nombre se puede cambiar desde la pestaña Prototipo (Figma, s.f.-a).

Aplicación de React Native al proyecto Rotula

Rotula móvil está hecha con Expo SDK 57, React Native 0.86 y React 19.2, y tiene seis pantallas: Login, Registro y cuatro pestañas (Inicio, Crear, Historial y Perfil). Se eligió el SDK 57 porque es la versión que abre el Expo Go instalado en el celular de prueba. La aplicación se probó en un teléfono Android con Expo Go y con una cuenta real de la web.

Arquitectura

El código separa la interfaz, el estado de la sesión y el acceso a la red. `App.js` carga las seis variantes de la fuente Figtree con `useFonts` y, mientras tanto, muestra una pantalla de carga; si las fuentes fallan, la aplicación sigue con la fuente del sistema. Después envuelve todo en `SafeAreaProvider`, que respeta las zonas seguras de la pantalla, y en `ProveedorSesion`, que comparte la sesión con todas las pantallas. La Tabla 2 resume qué hace cada parte.

Tabla 2

Estructura del código de Rotula móvil

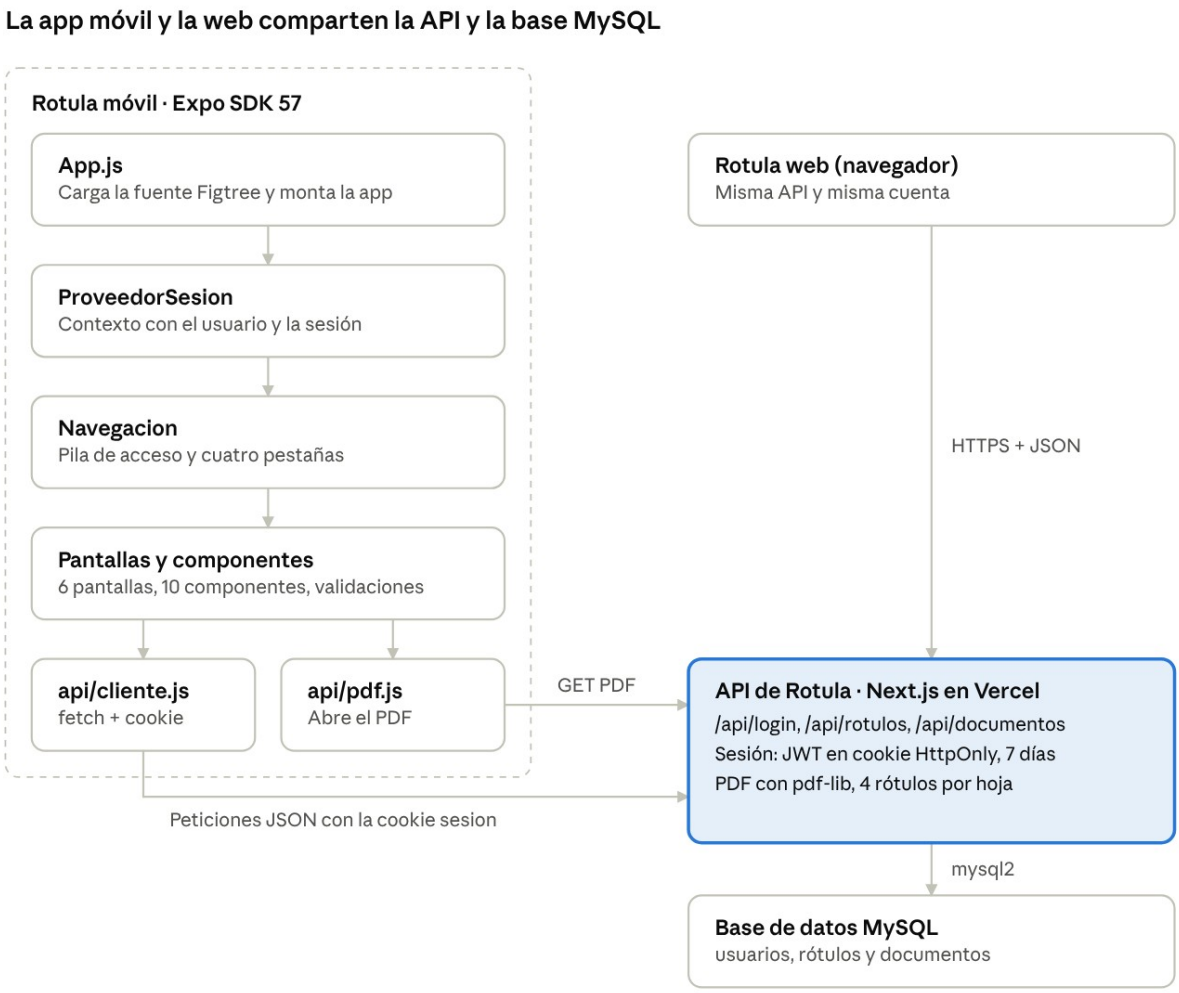
Archivo o carpeta	Responsabilidad
`App.js`	Carga las fuentes y monta el proveedor de sesión y la navegación
`src/contexto/Sesion.js`	Contexto con `usuario`, `revisando`, `iniciarSesion`, `registrar` y `cerrarSesion`
`src/navegacion/Navegacion.js`	Pila de acceso y pestañas principales
`src/pantallas/`	Las seis pantallas: Login, Registro, Inicio, CrearAfiche, Historial y Perfil
`src/componentes/`	Diez componentes reutilizables, como `Boton`, `Campo`, `CampoFecha`, `AvisoError`, `TarjetaDocumento` y `VistaPreviaAfiche`
`src/api/cliente.js`	Función `pedir`, que hace todas las peticiones a la API de Rotula
`src/api/pdf.js`	Descarga un PDF y lo abre con el menú de compartir del teléfono
`src/utilidades/`	Reglas de validación y formato de fechas
`src/tema.js`	16 colores, 6 pesos de la fuente Figtree y 3 radios de borde

Nota. Elaboración propia a partir del código del proyecto.

Las pantallas no llaman a `fetch` directamente: usan `pedir` o las funciones del contexto. Así, el manejo de la cookie, de los errores y de la dirección del servidor queda en un solo archivo. La Figura 1 muestra cómo se conectan las capas con la API y la base de datos que también usa la web.

Figura 1

Arquitectura de Rotula móvil y su conexión con la web



Nota. Elaboración propia a partir del código de la aplicación móvil y de la API web. La caja resaltada es la parte que comparten las dos aplicaciones.

Navegación

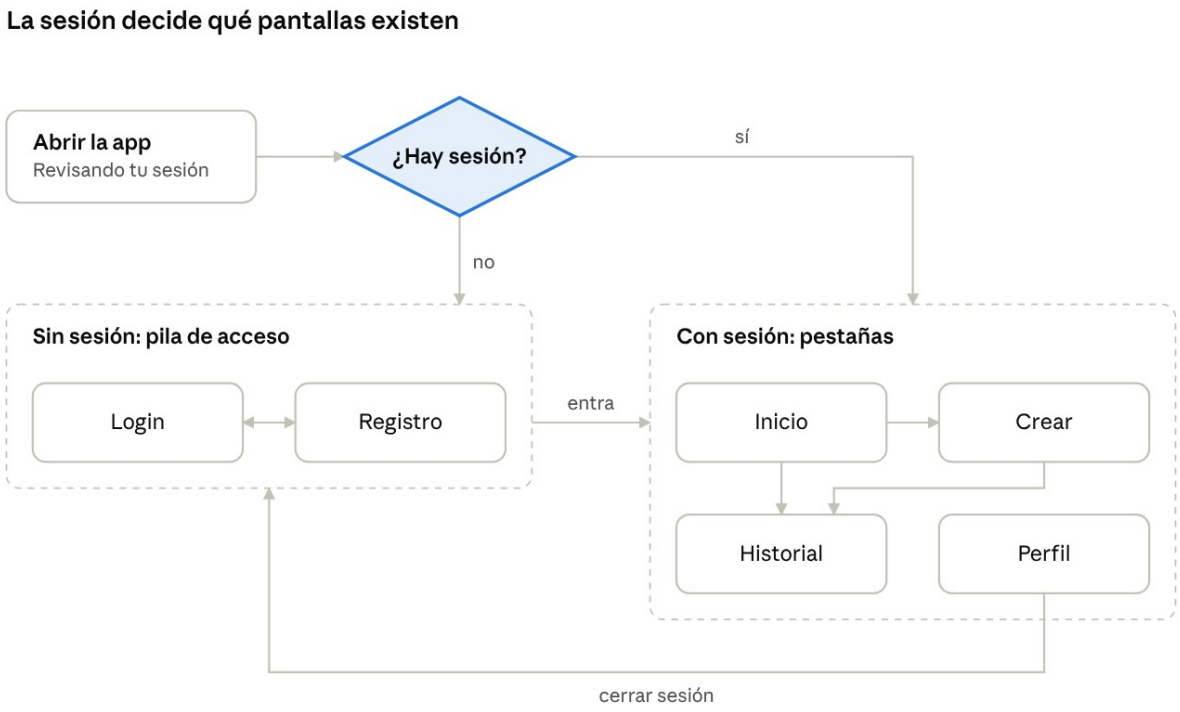
La navegación sigue el patrón de autenticación que recomienda React Navigation (s.f.-a). Un `NavigationContainer` con los colores de Rotula envuelve una pila creada con `createNativeStackNavigator`. Mientras el contexto revisa si hay una sesión guardada, la aplicación muestra el mensaje «Revisando tu sesión».

Si no hay usuario, la pila solo tiene Login y Registro, que se enlazan entre sí con `navigation.navigate`. Cuando el inicio de sesión funciona, el contexto guarda al usuario y la pila cambia sola a la pantalla `Principal`, que contiene las pestañas creadas con `createBottomTabNavigator`. Como Login y Registro dejan de existir en la pila, el botón atrás no puede volver a ellas; al cerrar sesión ocurre lo contrario.

Cada pestaña muestra el logo en el encabezado y un ícono de Ionicons que se ve relleno cuando está activa y de contorno cuando no. Desde Inicio, los botones «Crear afiches» y «Ver historial» llevan a sus pestañas, y el diálogo de PDF listo también lleva al historial. Inicio, Crear e Historial usan `useFocusEffect` para volver a pedir sus datos cada vez que se abren, de modo que un PDF recién generado aparece sin reiniciar la aplicación. La Figura 2 resume este flujo.

Figura 2

Flujo de navegación de Rotula móvil



Nota. Elaboración propia a partir de `src/navegacion/Navegacion.js`. Login y Registro se enlazan entre sí; al entrar desde cualquiera de las dos, la pila cambia a las pestañas. Inicio lleva a Crear y a Historial, y el diálogo de PDF listo de Crear también lleva a Historial. Cerrar sesión en Perfil devuelve a la pila de acceso.

Módulo principal

El módulo principal son las cuatro pestañas. La Tabla 3 describe lo que hace cada pantalla y los componentes de React Native que usa.

Tabla 3

Pantallas de Rotula móvil

Pantalla	Qué hace	Componentes destacados
Login	Pide correo y contraseña, valida y abre la sesión	`TextInput` con botón para mostrar la contraseña, `KeyboardAvoidingView`
Registro	Crea la cuenta y entra directo a la aplicación	`TextInput` en dos columnas para nombre y apellido
Inicio	Saludo con el nombre del negocio, rótulos usados del mes (de 10 gratis), ofertas activas, ofertas que vencen en 7 días y los últimos 3 PDF	`ScrollView` con `RefreshControl` para deslizar y recargar
Crear	Vista previa del afiche en vivo, formulario del producto, lista de rótulos pendientes y botón para generar el PDF	`Modal` con tres estados: confirmar, generando y documento listo; `Alert` para quitar un rótulo
Historial	Lista de todos los PDF generados; cada uno se descarga y se abre o comparte	`FlatList` con recarga y mensaje de lista vacía
Perfil	Iniciales, nombre, correo, negocio y plan; abre la web y cierra sesión con confirmación	`Linking`, `Alert`

Nota. Elaboración propia a partir del código del proyecto.

Validaciones

Las reglas están en `src/utilidades/validaciones.js`. Cada función recibe los datos del formulario y devuelve un objeto con un mensaje por cada campo que tiene un error; si el objeto está vacío, el formulario es válido y se envía. Las reglas repiten las que revisa la API para avisar antes de enviar, y el servidor vuelve a validar; por ejemplo, `/api/login` revisa el formato del correo y `/api/registro` exige 8 caracteres en la contraseña. La Tabla 4 resume las reglas.

Tabla 4

Reglas de validación de los formularios

Formulario	Campo	Regla
Login y Registro	Correo	Obligatorio, con formato válido y de máximo 120 caracteres
Login	Contraseña	Obligatoria
Registro	Nombre y apellido	Obligatorios, de 2 a 60 caracteres, solo letras (acepta tildes, ñ, apóstrofo, espacio y guion)
Registro	Negocio	Opcional, máximo 100 caracteres
Registro	Contraseña	Al menos 8 caracteres, con letras y al menos un número
Registro	Confirmación	Igual a la contraseña
Crear afiche	Código de barras	Solo números, de 8 a 14 dígitos
Crear afiche	Descripción	Obligatoria, al menos 3 caracteres
Crear afiche	Precio de oferta y precio normal	Formato como 1.99 (hasta dos decimales) y mayor a 0
Crear afiche	Copias	Número entero, al menos 1
Crear afiche	Fechas de inicio y fin	Las dos son obligatorias; la fecha fin no puede ser anterior a la de inicio

Nota. Elaboración propia a partir de `src/utilidades/validaciones.js`.

El componente `Campo` recibe el mensaje en la prop `error`: el borde cambia al color de error, el fondo se vuelve naranja claro, aparece un ícono de alerta y el mensaje queda debajo del campo. Cuando la persona vuelve a escribir, el error de ese campo se borra. Los errores del servidor se muestran en `AvisoError`, un aviso arriba del formulario marcado con `accessibilityRole="alert"` para los lectores de pantalla.

La aplicación traduce las respuestas del servidor a mensajes claros. Si las credenciales son incorrectas, Login muestra «El correo o la contraseña no son correctos». Si la API responde que el correo ya existe (código 409), Registro marca el campo de correo con «Ese correo ya tiene

una cuenta. Inicia sesión». Sin internet, `pedir` muestra «No hay conexión con el servidor. Revisa tu internet».

Además, la interfaz evita errores antes de que ocurran. Cada campo abre el teclado adecuado (`email-address`, `number-pad` o `decimal-pad`) y limita su largo con `maxLength`. Las fechas se eligen en el calendario nativo en lugar de escribirse, el botón «Generar PDF» se desactiva si la lista está vacía y la aplicación pide confirmación antes de quitar un rótulo o cerrar sesión. Mientras una petición está en curso, el botón muestra un indicador de carga y no se puede volver a presionar.

Estas decisiones siguen pautas de usabilidad conocidas. Krause (2019) recomienda poner el mensaje junto al campo, destacarlo con color rojo y un ícono, y no depender solo de un resumen al inicio del formulario. Nielsen (1994) propone prevenir los errores y, cuando ocurren, explicarlos en lenguaje sencillo y sugerir una solución, como hace el mensaje «Usa un precio válido. Ejemplo: 1.99». Una diferencia con Krause (2019) es que Rotula valida al presionar el botón y no al salir de cada campo; validar al salir del campo es una mejora posible para el siguiente avance.

Conexión con la API web

La dirección del servidor sale de la variable `EXPO_PUBLIC_API_URL`; si no existe, la aplicación usa https://rotula.vercel.app. Expo reemplaza estas variables por su valor al construir la aplicación y cualquier usuario puede verlas, por lo que no deben guardar secretos (Expo, 2026f). En Rotula móvil solo guardan la dirección pública de la API.

La función `pedir` llama a `fetch` con `credentials: "include"`, envía el cuerpo en JSON y convierte cualquier falla en un `ErrorApi` con el mensaje y el código HTTP. Trata como

error tanto una respuesta HTTP fallida como una respuesta con `ok: false`, porque `/api/login` responde con código 200 y `ok: false` cuando el correo no existe.

La sesión es la misma que usa la web. Al iniciar sesión, la API compara la contraseña con su hash de bcrypt, firma un JWT válido por 7 días y lo guarda en una cookie llamada `sesion` con el atributo `HttpOnly`. JavaScript no puede leer una cookie `HttpOnly` (MDN contributors, 2025b), así que la aplicación nunca maneja el token: la capa de red de React Native guarda la cookie y la reenvía en cada petición. El valor `include` de `credentials` pide que las credenciales, como las cookies, viajen siempre con la petición (MDN contributors, 2025a).

La documentación de React Native advierte que la autenticación con cookies todavía es inestable y que, en iOS, una cookie enviada junto con una redirección 302 no se guarda bien (Meta Platforms, 2026d). La API de Rotula responde en JSON y no usa redirecciones, y el flujo funcionó en las pruebas en Android. Queda pendiente probarlo en iOS.

La API de registro crea la cuenta pero no abre la sesión, por eso el contexto llama a `iniciarSesion` con los mismos datos justo después. Para los PDF, `POST /api/rotulos/pdf` genera el documento en el servidor con cuatro rótulos tamaño A6 por cada hoja A4. Para abrirlo, `abrirPdf` lo descarga con `fetch`, porque la ruta exige la cookie, lo guarda en la caché con la clase `File` de `expo-file-system` (Expo, 2026a) y abre el menú de compartir con `expo-sharing` (Expo, 2026c). Desde ese menú la persona lo ve, lo imprime o lo envía. La Tabla 5 lista las rutas que usa la aplicación.

Tabla 5

Rutas de la API de Rotula que usa la aplicación móvil

Método	Ruta	Uso
POST	`/api/login`	Iniciar sesión y recibir la cookie `sesion`
POST	`/api/registro`	Crear la cuenta
POST	`/api/logout`	Cerrar la sesión
GET	`/api/usuario`	Datos del usuario de la sesión
GET y POST	`/api/rotulos`	Listar y agregar rótulos
DELETE	`/api/rotulos/:id`	Quitar un rótulo de la lista
POST	`/api/rotulos/pdf`	Generar el PDF con los rótulos pendientes
GET	`/api/documentos`	Historial de PDF
GET	`/api/documentos/:uuid/pdf`	Descargar un PDF

Nota. Elaboración propia a partir del código de la aplicación móvil y de la API web.

Por qué se usó la API MySQL existente en lugar de Firebase o SQLite

Firebase y SQLite son las opciones de datos que se ven en la materia. Firebase Authentication ofrece servicios de backend, SDK y bibliotecas de interfaz para iniciar sesión con correo, teléfono o proveedores como Google (Google, 2026a). Cloud Firestore es una base de datos NoSQL en la nube, con soporte sin conexión y actualizaciones en tiempo real (Google, 2026b). Por su parte, `expo-sqlite` le da a la aplicación una base de datos local que se conserva entre reinicios (Expo, 2026d).

Rotula móvil usa la API existente por tres razones:

1. **Una sola fuente de datos.** Rotula ya guarda en MySQL los usuarios, los rótulos y los documentos de la web en producción. Con la misma API, una cuenta creada en la web sirve en el celular y al revés, y un PDF generado en un lado aparece en el historial del otro.

2. **La lógica ya está en el servidor.** Las contraseñas se guardan con bcrypt, la sesión usa JWT y el PDF se genera en el servidor con la biblioteca pdf-lib. Con Firebase habría que migrar las cuentas y repetir esa lógica; con SQLite los datos quedarían en un solo teléfono y el PDF tendría que generarse en el dispositivo.
3. **Ningún secreto en la aplicación.** La aplicación no guarda claves ni credenciales de la base de datos; solo conoce la dirección pública de la API.

La desventaja es que la aplicación necesita internet para todo. SQLite podría usarse más adelante como caché local, por ejemplo para guardar la lista de rótulos sin conexión.

Proceso de diseño UX/UI en Figma

El diseño de Rotula móvil parte de la web que ya existía. La web tenía una identidad definida en `app/estilos.css`: la fuente Figtree, una escala de grises neutros y un naranja de acento. La aplicación móvil se construyó con esa misma identidad y, después, se documentó en Figma. Es decir, los mockups no se hicieron en una etapa anterior al código: se armaron a partir de la aplicación y de la web para tener una referencia visual única para los siguientes avances.

El archivo de Figma se llama «Rotula móvil · Mockups Avance 1» (enlace en la introducción) y tiene una sola página, «Rotula móvil». Esa página contiene la sección «Pantallas», con los ocho mockups, y el marco «Componentes», con el sistema de diseño. El proceso siguió estos pasos:

1. Revisar la web: sus colores, su tipografía y los textos de las páginas de inicio de sesión, registro e inicio.
2. Pasar esos valores al archivo `src/tema.js` de la aplicación y construir las pantallas en React Native.
3. Crear en Figma las variables y los estilos de texto con los mismos valores del código.

4. Crear los componentes y sus variantes a partir de los componentes de React Native.
5. Armar las ocho pantallas con instancias de esos componentes.
6. Conectar las pantallas en un prototipo navegable.

Sistema de diseño

El archivo tiene una colección de variables llamada «Rotula» con 16 colores y 8 medidas de espaciado y radio. Los colores son los mismos 16 de `src/tema.js`, y la mayoría viene de la paleta de la web. La Tabla 6 muestra una selección y su origen.

Tabla 6

Selección de variables de Figma, su valor en el código y su origen en la web

Variable en Figma	Valor	Nombre en `tema.js`	Origen en `estilos.css`
`color/naranja`	#E2502A	`naranja`	`--color-accent2-500`
`color/naranja-oscuro`	#C84320	`naranjaOscuro`	`--color-accent2-600`
`color/naranja-claro`	#FDF1EC	`naranjaClaro`	`--color-accent2-100`
`color/error`	#A5361A	`error`	`--color-accent2-700`
`color/texto`	#111111	`texto`	`--color-accent-500`
`color/texto-suave`	#5C5C58	`textoSuave`	`--color-neutral-700`
`color/borde`	#E6E6E3	`borde`	`--color-neutral-300`
`color/superficie`	#F7F7F5	`superficie`	`--color-surface`
`color/amarillo-afiche`	#F2C94C	`amarilloAfiche`	Propio de la app (vista previa del afiche)
`radio/input`	12	`radios.input`	No aplica
`radio/tarjeta`	16	`radios.tarjeta`	No aplica
`radio/pildora`	999	`radios.pildora`	No aplica
`espacio/lg`	20	Margen de las pantallas con pestañas	No aplica

Nota. Elaboración propia a partir del archivo de Figma, de `src/tema.js` y de `app/estilos.css`.

La tipografía se definió con 17 estilos de texto de la familia Figtree. Cada estilo repite un tamaño del código: por ejemplo, Botón/Base es Figtree Bold de 16 px, Pestaña es Figtree SemiBold de 12 px y Número/Dato es Figtree ExtraBold de 36 px con interlineado de 40 px, igual que los números de las tarjetas de Inicio.

Componentes

El marco «Componentes» reúne las piezas que se repiten en las pantallas (Figura 3). Hay 22 íconos, que corresponden a los íconos de Ionicons que usa el código (casa, reloj, calendario, documento, basura y otros). Los componentes con variantes reflejan las props de sus equivalentes en React Native, como muestra la Tabla 7.

Tabla 7

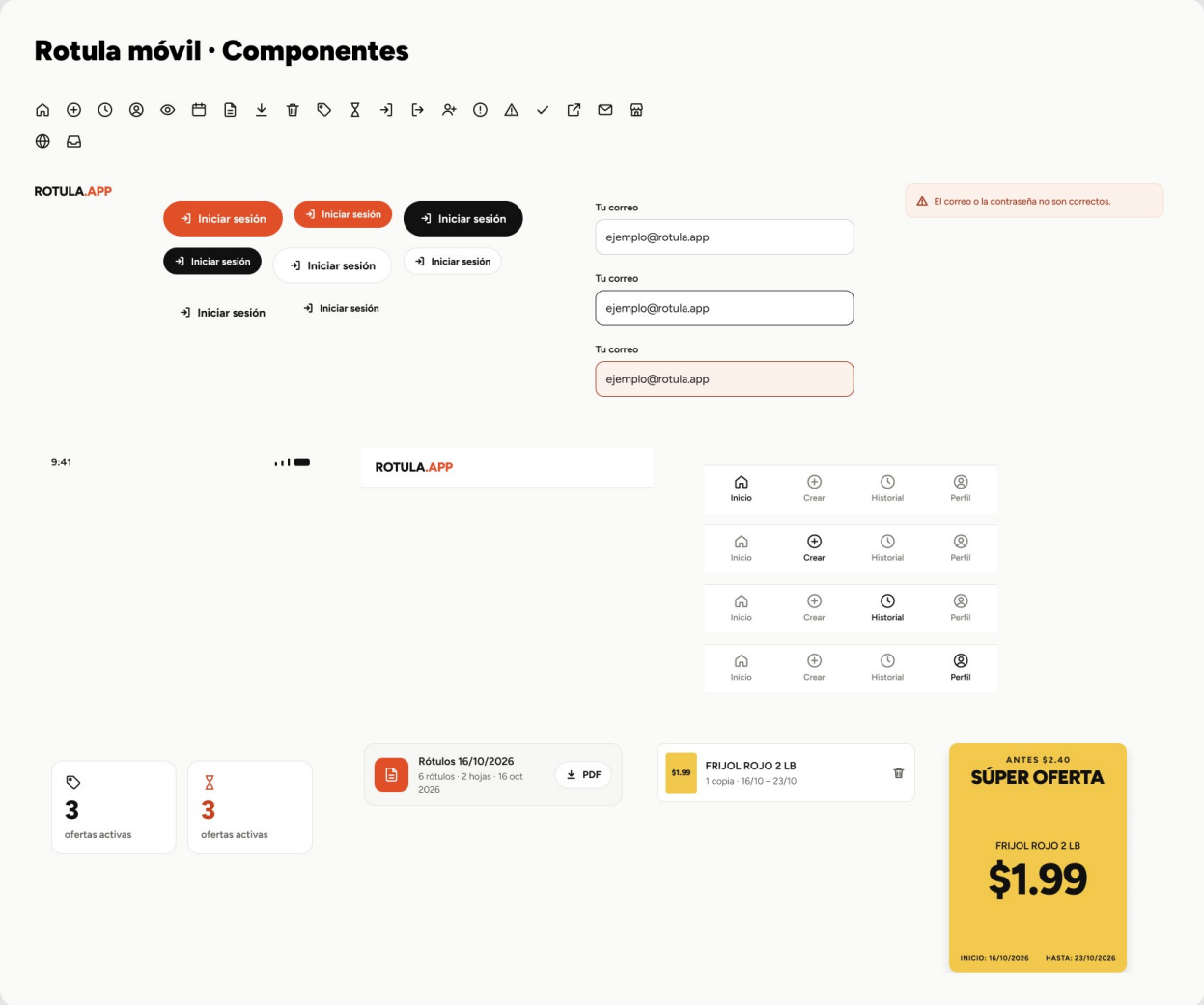
Componentes de Figma y su equivalente en el código

Componente en Figma	Propiedades y valores	Componente en React Native
Botón	Estilo = Acento, Primario, Secundario o Fantasma; Tamaño = Normal o Pequeño (8 variantes)	`Boton`, con las props `variante` y `pequeno`
Campo	Estado = Normal, Foco o Error	`Campo`, que cambia el borde según el foco y la prop `error`
Barra de pestañas	Activa = Inicio, Crear, Historial o Perfil	Barra de `createBottomTabNavigator`
Tarjeta dato	Tono = Normal o Alerta	Tarjetas de ofertas activas y por vencer en Inicio
Aviso de error	Sin variantes	`AvisoError`
Tarjeta documento	Sin variantes	`TarjetaDocumento`
Ítem de lista	Sin variantes	Fila de la lista de rótulos en Crear
Vista previa afiche	Sin variantes	`VistaPreviaAfiche`
Logo, Encabezado y Barra de estado	Sin variantes	`Logo` y encabezado de las pestañas

Nota. Elaboración propia a partir del archivo de Figma y del código del proyecto.

Figura 3

Marco «Componentes» del archivo de Figma



Nota. Captura del archivo «Rotula móvil» en Figma. Elaboración propia.

Pantallas

La sección «Pantallas» tiene ocho marcos de 412 px de ancho, el tamaño de un teléfono Android. La mayoría mide 917 px de alto; Registro mide 943 px y Crear afiches 1569 px porque muestran todo el contenido que en el celular se recorre con desplazamiento. Las pantallas están hechas con instancias de los componentes y usan datos de ejemplo, como la tienda ficticia «Tienda La Esquina». Las medidas repiten las del código: en el formulario de Login, los campos

tienen 24 px de margen y 18 px de separación, igual que el estilo `formulario` de `MarcoAcceso`.

La Tabla 8 describe cada marco y las Figuras 4 a 6 los muestran.

Tabla 8

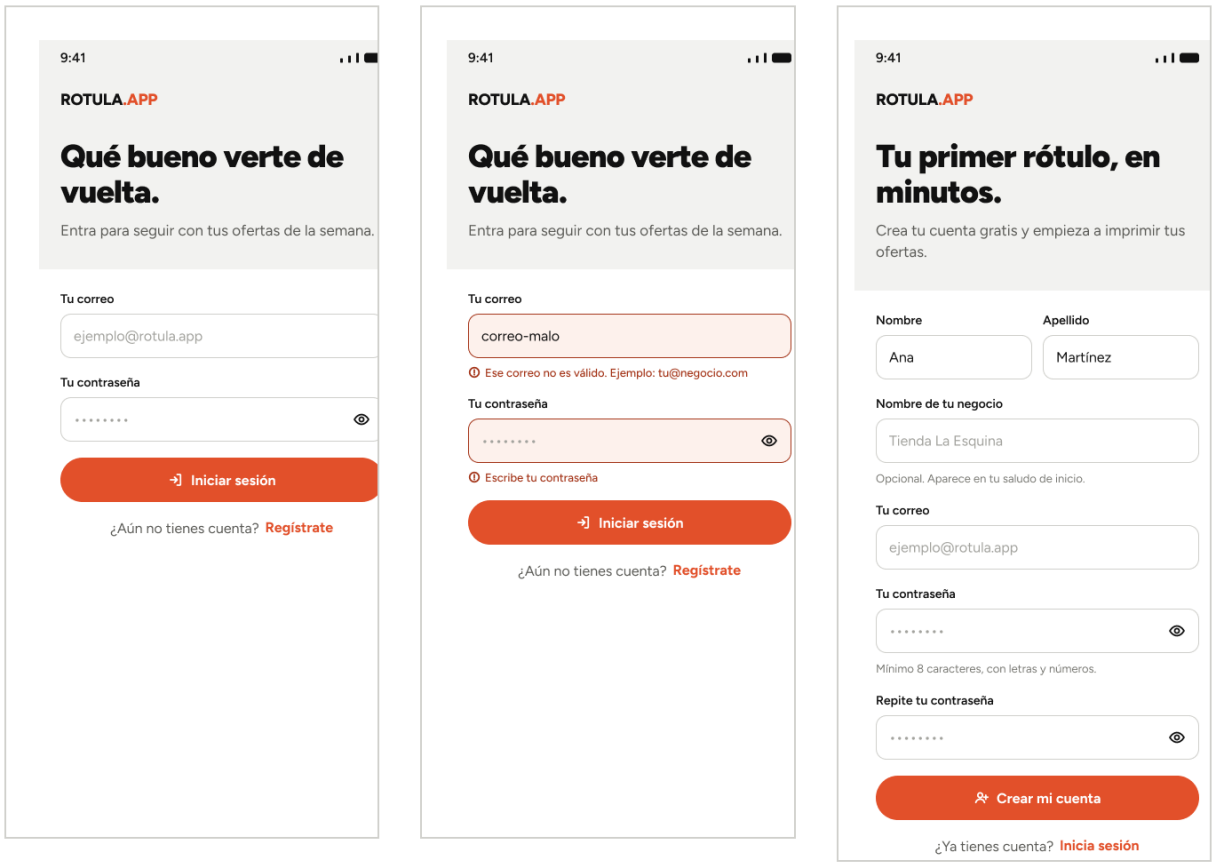
Marcos de la sección «Pantallas»

Marco	Contenido
01 · Login	Logo, el saludo «Qué bueno verte de vuelta.», campos de correo y contraseña, botón «Iniciar sesión» y enlace «Regístrate»
02 · Login · Validaciones	La misma pantalla con los dos campos en estado de error y su mensaje debajo
03 · Registro	Nombre y apellido en dos columnas, negocio, correo, contraseña con texto de ayuda, confirmación, botón «Crear mi cuenta» y enlace «Inicia sesión»
04 · Inicio	Fecha, saludo con el nombre del negocio, uso del mes, botones «Crear afiches» y «Ver historial», dos tarjetas de dato, dos documentos recientes y la barra de pestañas
05 · Crear afiches	Vista previa del afiche, formulario completo, lista con dos productos y botón para generar el PDF
06 · Crear · PDF listo (Modal)	Fondo oscuro sobre Crear y el diálogo «Documento listo» con los botones «Abrir PDF», «Ver historial» y «Cerrar»
07 · Historial	Título, descripción y cuatro tarjetas de documento
08 · Perfil	Avatar con iniciales, nombre, negocio, datos de la cuenta y los botones para abrir la web y cerrar sesión

Nota. Elaboración propia a partir del archivo de Figma.

Figura 4

Mockups de acceso: Login, Login con validaciones y Registro

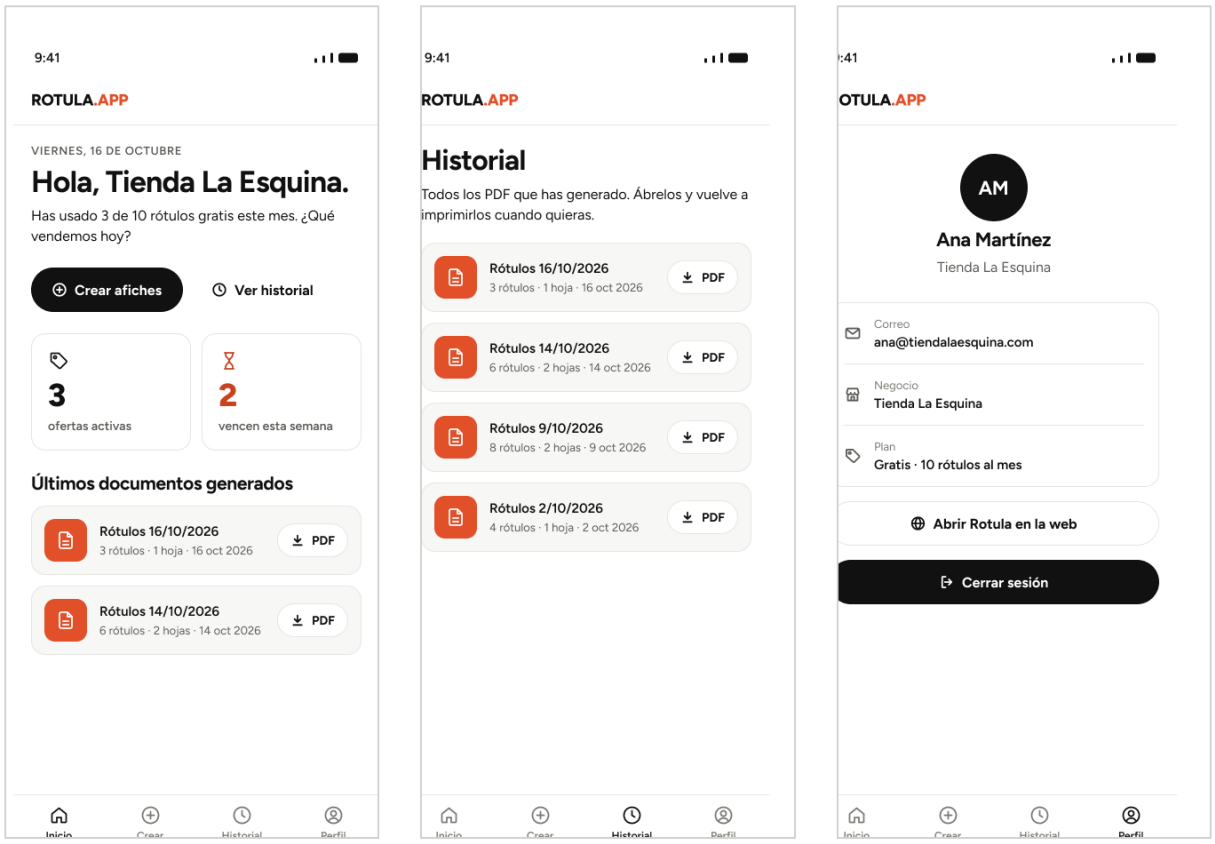


01 · Login 02 · Login · Validaciones 03 · Registro

Nota. Exportación del archivo «Rotula móvil» en Figma. Elaboración propia.

Figura 5

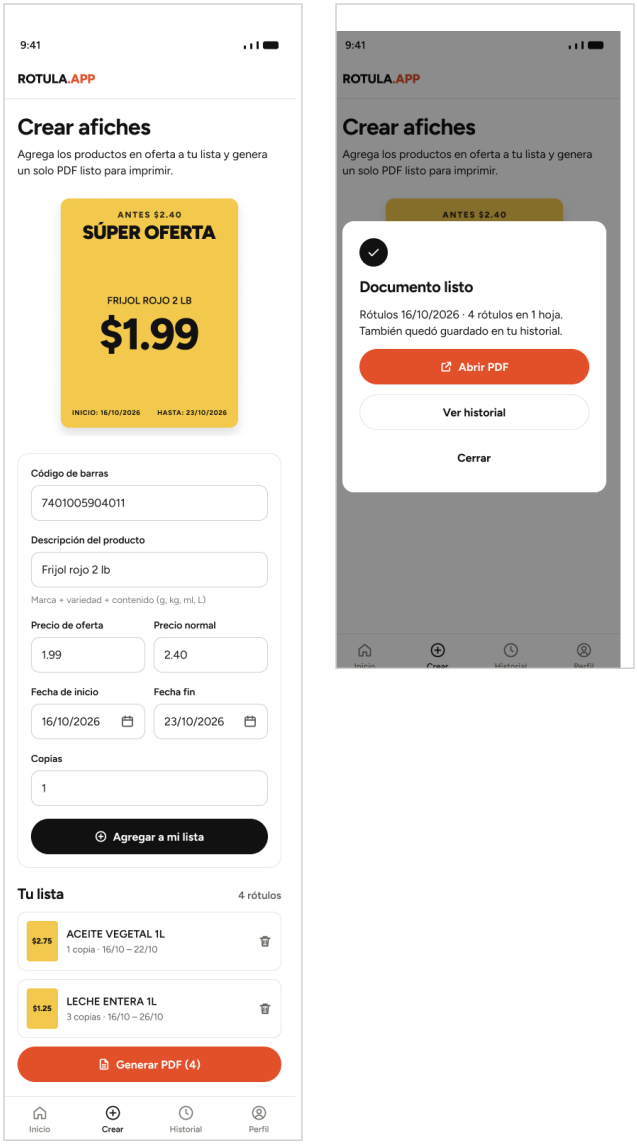
Mockups del módulo principal: Inicio, Historial y Perfil



Nota. Exportación del archivo «Rotula móvil» en Figma. Elaboración propia.

Figura 6

Mockups de Crear afiches y del diálogo de PDF listo



Nota. Exportación del archivo «Rotula móvil» en Figma. Elaboración propia. El marco 05 muestra la pantalla completa, que en el celular se recorre con desplazamiento.

Prototipo navegable

El prototipo tiene 32 interacciones y un flujo llamado «Rotula móvil», cuyo punto de inicio es el marco 01 · Login. Las interacciones conectan los botones, los enlaces y las pestañas

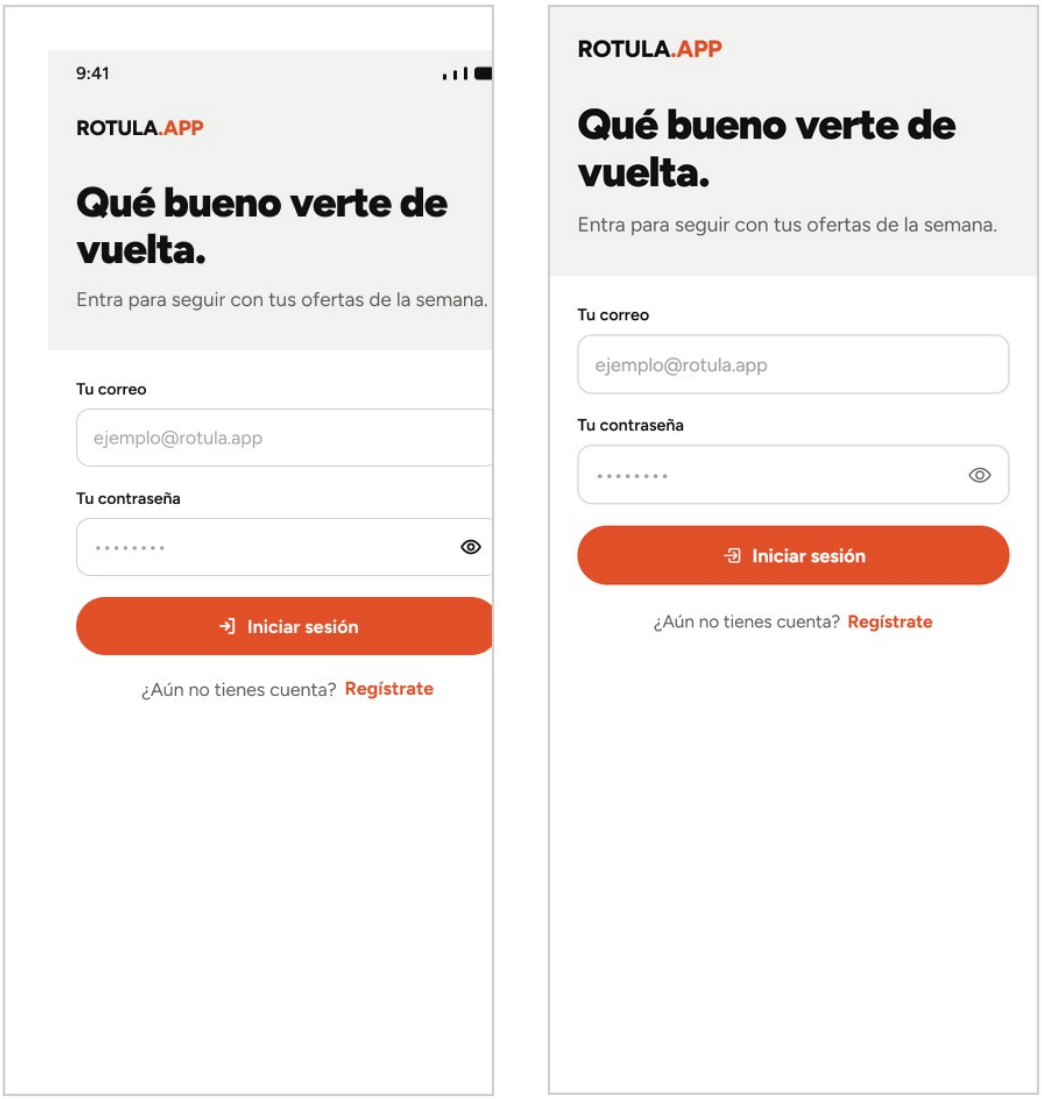
de los marcos para recorrer la aplicación desde el inicio de sesión, siguiendo la navegación de la Figura 2. El prototipo se abre desde el archivo de Figma con el botón Presentar.

Coherencia entre el diseño y la aplicación

La Figura 7 compara el mockup de Login con la misma pantalla de la aplicación. La estructura, los textos, la fuente, los colores y las medidas coinciden. La única diferencia es la barra de estado, que en Figma forma parte del marco y en el teléfono la dibuja el sistema.

Figura 7

Comparación entre el mockup de Login y la pantalla Login de la aplicación



Nota. Elaboración propia. A la izquierda, el marco 01 · Login de Figma; a la derecha, la pantalla Login de la aplicación ejecutada con Expo en el navegador (react-native-web) a 412 × 917 px.

Decisiones de experiencia de usuario

Varias decisiones del diseño responden a las heurísticas de usabilidad de Nielsen (1994):

- **Visibilidad del estado del sistema.** Los botones muestran un indicador mientras esperan al servidor, la pantalla de carga dice qué está pasando («Revisando tu sesión», «Pintando tus rótulos») y, al agregar un producto, aparece un mensaje como «Listo, se agregó 1 rótulo a tu lista».
- **Relación con el mundo real.** La vista previa del afiche se actualiza mientras la persona llena el formulario, así que ve lo que se va a imprimir antes de generar el PDF. Los textos usan el lenguaje del negocio: oferta, precio normal, copias y rótulos.
- **Consistencia.** La aplicación repite los colores, la fuente y los textos de la web; por ejemplo, el saludo «Qué bueno verte de vuelta.» del login es el mismo en las dos.
- **Prevención y recuperación de errores.** Las validaciones, los teclados adecuados y las confirmaciones descritas en la sección de validaciones evitan errores comunes y explican cómo corregirlos.
- **Estados vacíos.** Cuando no hay datos, la pantalla dice qué hacer, por ejemplo «Tu lista está vacía. Agrega un producto con el formulario de arriba».

Para la accesibilidad, los campos y botones tienen `accessibilityLabel` y `accessibilityRole`, y los íconos pequeños, como el ojo de la contraseña y la basura, amplían su área táctil con `hitSlop`.

Conclusiones

React Native y Expo permitieron construir una aplicación para Android e iOS con una sola base de código en JavaScript, usando los mismos conceptos de React que ya usa la web: componentes, props, estado, contexto y efectos. El flujo de inicio de sesión con React Navigation quedó simple porque sigue el patrón que recomienda su documentación: las pantallas dependen del estado de la sesión y no hace falta navegar a mano entre el acceso y el módulo principal.

Usar la API y la base MySQL de la web fue la decisión con más efecto en el proyecto. Una misma cuenta y los mismos datos sirven en las dos plataformas, la aplicación no guarda secretos y la lógica de seguridad y de generación de PDF no se repite. El costo es que la aplicación necesita internet y depende de una sesión por cookie, que la documentación de React Native todavía considera inestable; por eso falta probarla en iOS.

Las validaciones en el cliente, con mensajes junto a cada campo y en lenguaje sencillo, evitan peticiones inútiles y explican cómo corregir cada error, mientras que el servidor sigue validando por su cuenta. La siguiente mejora es validar cada campo al terminar de escribirlo, como recomiendan las pautas de usabilidad consultadas.

En Figma, el sistema de diseño (16 colores, 8 medidas, 17 estilos de texto y componentes con variantes) quedó con los mismos nombres y valores que el código. Esa correspondencia uno a uno hace fácil revisar que la aplicación y los mockups coincidan. Como los mockups se hicieron a partir de la aplicación y de la web, sirvieron para documentar y verificar el diseño más que para explorar alternativas. Para los siguientes avances conviene invertir el orden: diseñar primero las pantallas nuevas en Figma, probar el prototipo con dueños de tiendas y después programarlas.

Referencias

Biørn-Hansen, A., Grønli, T.-M., y Ghinea, G. (2018). A survey and taxonomy of core concepts and research challenges in cross-platform mobile development. *ACM Computing Surveys*, *51*(5), 1–34. https://doi.org/10.1145/3241739

Expo. (s.f.). *Expo Go*. Recuperado el 16 de octubre de 2026, de https://expo.dev/go

Expo. (2026a, 29 de junio). *Expo FileSystem*. Expo Documentation. https://docs.expo.dev/versions/v57.0.0/sdk/filesystem/

Expo. (2026b, 17 de agosto). *Set up your environment*. Expo Documentation. https://docs.expo.dev/get-started/set-up-your-environment/

Expo. (2026c, 21 de agosto). *Expo Sharing*. Expo Documentation. https://docs.expo.dev/versions/v57.0.0/sdk/sharing/

Expo. (2026d, 21 de agosto). *Expo SQLite*. Expo Documentation. https://docs.expo.dev/versions/v57.0.0/sdk/sqlite/

Expo. (2026e, 17 de septiembre). *Core concepts*. Expo Documentation. https://docs.expo.dev/core-concepts/

Expo. (2026f, 12 de octubre). *Environment variables in Expo*. Expo Documentation. https://docs.expo.dev/guides/environment-variables/

Figma. (s.f.-a). *Crear y gestionar de flujos de prototipos*. Figma Learn - Centro de ayuda. Recuperado el 16 de octubre de 2026, de https://help.figma.com/hc/es-es/articles/360039823894

Figma. (s.f.-b). *Create and apply text styles*. Figma Learn - Help Center. Recuperado el 16 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/360039957034

Figma. (s.f.-c). *Create and use variants*. Figma Learn - Help Center. Recuperado el 16 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/360056440594

Figma. (s.f.-d). *Descripción general de las variables, las colecciones y los modos*. Figma Learn - Centro de ayuda. Recuperado el 16 de octubre de 2026, de https://help.figma.com/hc/es-es/articles/14506821864087

Figma. (s.f.-e). *Guía de componentes en Figma*. Figma Learn - Centro de ayuda. Recuperado el 16 de octubre de 2026, de https://help.figma.com/hc/es-es/articles/360038662654

Figma. (s.f.-f). *Guía de disposición automática*. Figma Learn - Centro de ayuda. Recuperado el 16 de octubre de 2026, de https://help.figma.com/hc/es-es/articles/360040451373

Figma. (s.f.-g). *Guía para la creación de prototipos en Figma*. Figma Learn - Centro de ayuda. Recuperado el 16 de octubre de 2026, de https://help.figma.com/hc/es-es/articles/360040314193

Figma. (s.f.-h). *Marcos en Figma Design*. Figma Learn - Centro de ayuda. Recuperado el 16 de octubre de 2026, de https://help.figma.com/hc/es-es/articles/360041539473

Figma. (s.f.-i). *¿Qué es Figma?* Figma Learn - Centro de ayuda. Recuperado el 16 de octubre de 2026, de https://help.figma.com/hc/es-es/articles/14563969806359

Google. (2026a, 14 de octubre). *Firebase Authentication*. Firebase. https://firebase.google.com/docs/auth

Google. (2026b, 15 de octubre). *Cloud Firestore*. Firebase. https://firebase.google.com/docs/firestore

Krause, R. (2019, 3 de febrero). *10 design guidelines for reporting errors in forms*. Nielsen Norman Group. https://www.nngroup.com/articles/errors-forms-design-guidelines/

MDN contributors. (2025a, 19 de mayo). *Request: credentials property*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Request/credentials

MDN contributors. (2025b, 7 de julio). *HTTP cookies*. MDN Web Docs. https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Cookies

Meta Platforms. (s.f.-a). *Lifecycle of reactive effects*. React. Recuperado el 16 de octubre de 2026, de https://react.dev/learn/lifecycle-of-reactive-effects

Meta Platforms. (s.f.-b). *Passing data deeply with context*. React. Recuperado el 16 de octubre de 2026, de https://react.dev/learn/passing-data-deeply-with-context

Meta Platforms. (s.f.-c). *Writing markup with JSX*. React. Recuperado el 16 de octubre de 2026, de https://react.dev/learn/writing-markup-with-jsx

Meta Platforms. (2026a, 12 de agosto). *Core components and native components*. React Native. https://reactnative.dev/docs/intro-react-native-components

Meta Platforms. (2026b, 12 de agosto). *Get started with React Native*. React Native. https://reactnative.dev/docs/environment-setup

Meta Platforms. (2026c, 12 de agosto). *React fundamentals*. React Native. https://reactnative.dev/docs/intro-react

Meta Platforms. (2026d, 14 de octubre). *Networking*. React Native. https://reactnative.dev/docs/network

Nielsen, J. (1994, 24 de abril). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

React Navigation. (s.f.-a). *Authentication flows*. Recuperado el 16 de octubre de 2026, de https://reactnavigation.org/docs/auth-flow/

React Navigation. (s.f.-b). *Bottom tabs navigator*. Recuperado el 16 de octubre de 2026, de https://reactnavigation.org/docs/bottom-tab-navigator/

React Navigation. (s.f.-c). *Getting started*. Recuperado el 16 de octubre de 2026, de https://reactnavigation.org/docs/getting-started/

React Navigation. (s.f.-d). *Native stack navigator*. Recuperado el 16 de octubre de 2026, de https://reactnavigation.org/docs/native-stack-navigator/

React Navigation. (s.f.-e). *useFocusEffect*. Recuperado el 16 de octubre de 2026, de https://reactnavigation.org/docs/use-focus-effect/